

AI 使用说明情况（第二次挑战）

1. 使用的 AI 模型

本次挑战使用 Codex（GPT-6）作为学习和代码协作工具，辅助理解前缀缓存机制、排查环境问题、编写实验脚本及整理报告。实验数据来自实际运行的 SGLang 服务，源码分析和结论以对应版本源码及逐请求记录为依据。

2. 使用过的提示词类型

以下是按实际任务整理的提示词示例，用于展示提问方向，并非逐字保存的聊天记录：

- “请结合因果注意力和位置编码，解释相同前缀为什么可以复用 K/V，命中后还需要计算哪些 token。”
- “请根据 SGLang 0.5.14、Ray 2.56.0 和当前环境，给出服务启动问题的排查思路。”
- “请协助设计两组输入长度一致的前缀缓存负载，并检查预热、清缓存和并发控制逻辑。”
- “请沿原生 /generate 接口梳理请求处理流程，标出关键函数和所在文件。”
- “请核对逐请求记录与汇总表，检查 TTFT、TPOT、缓存命中率和实际 Prefill token 的统计口径。”
- “请检查报告中的技术表述和交付结构。”

提问时结合具体版本、报错信息、源码片段和实验结果，使讨论围绕当前任务展开。

3. AI 如何帮助学习，以及在哪些环节使用

AI 的辅助作用主要体现在四个环节。第一，在概念理解上，帮助区分 Prefill、Decode 和前缀 K/V 复用，将注意力公式与 TTFT、TPOT 的变化联系起来。第二，在环境排查上，提供 Windows 客户端与 WSL 服务之间的检查思路。第三，在实验与源码阅读上，协助编写负载生成、流式记录和校验脚本，并提供关键符号的检索线索。第四，在材料整理上，协助绘制流程图、核对报告表格、修订技术表述及个人感受的表达。

这些帮助使问题更容易拆解。例如，阅读请求流程时，可以分别追问“请求如何入队、缓存如何匹配、哪些 token 需要前向计算、KV 如何写回、结果如何返回”。AI 提供解释和候选路径，是否符合本次运行则需要结合源码和数据核对。

4. 是否出现过 AI 误导，以及如何验证

出现过需要纠正的情况。初版负载设计依据 PDF 文字提取，遗漏了表格中的长度要求，曾使用 128 token 前缀；重新查看原题页面后，改为 2048 token 公共前缀、64 token 后缀及 2112 token 总输入。环境讨论中也曾误判 Windows Python 环境的适用性，随后根据已有 WSL 环境和版本信息调整服务启动方案。

技术解释也需要核验。例如，默认 FCFS 路径中的前缀匹配由 Req.init_next_round_input 发起，不能把 match_prefix_for_req 画成每条请求必经的节点；生成 16 token 时，Prefill 产生首个输出 token，随后通常执行 15 步 Decode。报告已据此修正。

本次通过核对题目页面、v0.5.14 源码、逐请求 SSE 记录及汇总指标验证这些解释。对于 TPOT 的变化，只将 Prefill 与 Decode 资源争用作为可能原因，没有把一轮对照结果当作确定的因果证明。这让我认识到，AI 能辅助提出思路、编写代码和整理表达，但其回答仍需经过版本核对与实验验证。